SIMATS ENGINEERING

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

CHENNAI — 602105

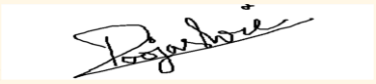
Department of Computer Science and Engineering

ASSESSMENT TOOL 2 — SCHEMA ANALYSIS AND VIVA VOCE

Course Code:	DSA05	Course Title:	Query Processing for Data Science With Real Time Applications
CO Assessed:	CO2	Assessment:	Tool 2 — Schema Analysis and Viva Voce
Weightage:		Total Marks:	20 Marks
GO2 Statement: Use Python basics and SQL libraries to connect to databases SQLite, MySQL, PostgreSQL and perform CRUD operations like Create, Read, Update, Delete. (BL6)			
Student Name:	R.S.Poojashri	Reg. No.:	192324266
Section / Batch:	B.Tech(AI&Ds)	Date of Submission:	03-08-2026
Faculty charge:	Dr. S. Ponmaniraj	Project Mode:	Individual Submission

Code of Conduct

I R.S.Poojashri (192324266) (Name / Reg No) certify that this submission is my original work and that I have adhered to all guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature: 

Identification of Entities and Attributes (4 Marks)	Understanding of Relationships (4 Marks)	Analysis of Keys and Constraints (4 Marks)	Detection of Design Issues (4 Marks)	Viva Voce (4 Marks)	Total (20 Marks)

1. University Course Registration System

A university database contains the following tables:

- Student(StudentID, Name, DepartmentID)
- Department(DepartmentID,DepartmentName)
- Course(CourseID, CourseName, FacultyID)
- Faculty(FacultyID,FacultyName)
- Enrollment(StudentID, CourseID,Semester)

A student can enroll in multiple courses, and each course can have multiple students.

Answers :

1. Identification of Entities and Attributes (4 Marks)

Entities

- Student (StudentID, Name, DepartmentID)
- Department (DepartmentID, DepartmentName)
- Course (CourseID, CourseName, FacultyID)
- Faculty (FacultyID, FacultyName)
- Enrollment (StudentID, CourseID, Semester)

2. Understanding of Relationships (4 Marks)

- One Department → Many Students (1:M)
- One Faculty → Many Courses (1:M)
- Student ↔ Course (M:N) through Enrollment

3. Analysis of Keys and Constraints (4 Marks)

Primary Keys

- StudentID
- DepartmentID
- CourseID
- FacultyID
- (StudentID, CourseID, Semester) in Enrollment

Foreign Keys

- DepartmentID → Department
- FacultyID → Faculty

- StudentID → Student
- CourseID → Course

4. Detection of Design Issues (4 Marks)

- Enrollment is necessary for the many-to-many relationship.
- Add Grade field if grades need to be stored.

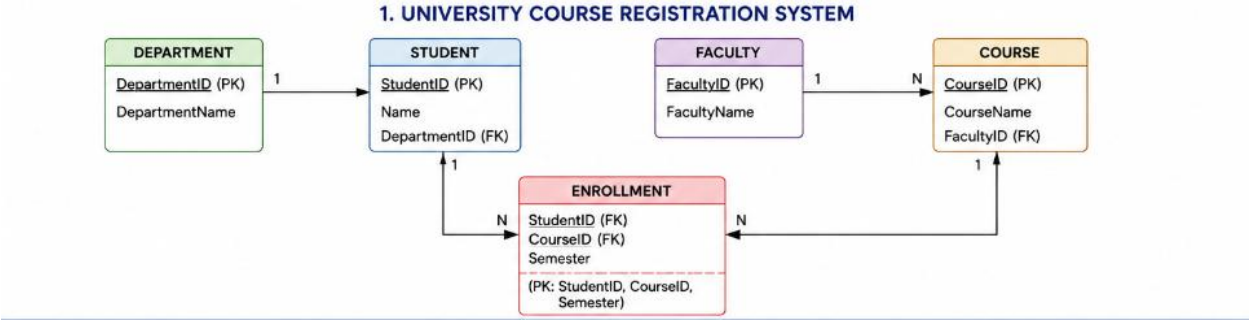
5. Viva Questions

Q1. Why is the Enrollment table necessary instead of storing CourseID directly in the Student table?

- A student can enroll in many courses and a course can have many students.
- This is a **many-to-many relationship**.
- The Enrollment table stores each student-course combination and avoids data duplication.

Q2. If the university wants to track grades for each enrolled course, how would you modify the schema?

- Add a **Grade** attribute to the Enrollment table.
- Enrollment(StudentID, CourseID, Semester, Grade)



Why this schema?

We use **five tables** because each represents a different real-world object (entity):

- **Student** stores student information.
- **Department** stores department details only once.
- **Faculty** stores faculty information.
- **Course** stores course details.
- **Enrollment** connects Students and Courses.

The **Enrollment** table is required because:

- One student can enroll in many courses.
- One course can have many students.
- This is a **Many-to-Many (M:N)** relationship.

- A bridge table (Enrollment) removes redundancy and follows normalization.

2. Hospital Management System

A hospital database contains:

- Patient(PatientID, Name, Age)
- Doctor(DoctorID, DoctorName, Specialization)
- Appointment(AppointmentID, PatientID, DoctorID, Date)
- Prescription (PrescriptionID, AppointmentID, Medicine)

Multiple patients can consult the same doctor.

Answers:

1. Identification of Entities and Attributes

- Patient (PatientID, Name, Age)
- Doctor (DoctorID, DoctorName, Specialization)
- Appointment (AppointmentID, PatientID, DoctorID, Date)
- Prescription (PrescriptionID, AppointmentID, Medicine)

2. Understanding of Relationships

- One Patient → Many Appointments
- One Doctor → Many Appointments
- One Appointment → One or Many Prescriptions

3. Analysis of Keys and Constraints

Primary Keys

- PatientID
- DoctorID
- AppointmentID
- PrescriptionID

Foreign Keys

- PatientID → Patient
- DoctorID → Doctor
- AppointmentID → Appointment

4. Detection of Design Issues

- Multiple medicines cannot be stored efficiently.
- Create a separate Medicine table.

5. Viva Questions

Q1. What problems might occur if prescription details are stored directly in the Appointment table?

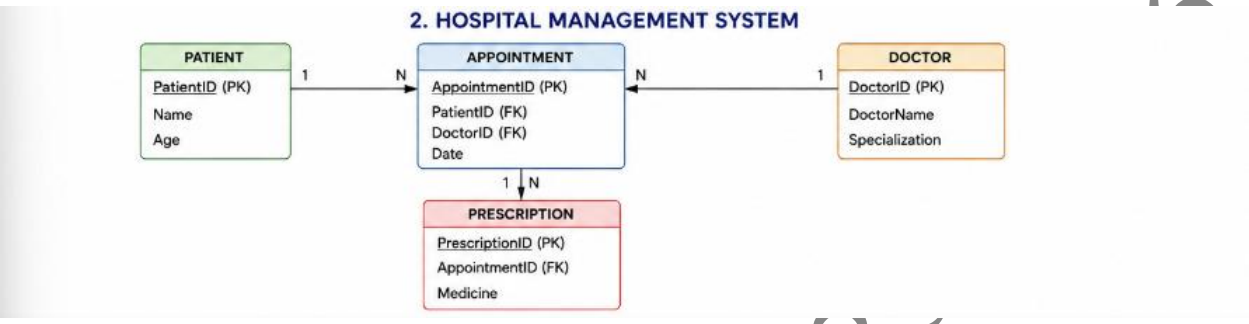
- Only one medicine can be stored easily.
- Data becomes repetitive.

- Difficult to manage multiple medicines.
- Violates database normalization.

Q2. How would the schema change if a prescription contains multiple medicines?

Create another table:

Prescription(PrescriptionID, AppointmentID) PrescriptionMedicine(PrescriptionID, MedicineName, Dosage)



Why this schema?

Each table stores a different type of information.

- Patient stores patient details.
- Doctor stores doctor details.
- Appointment connects patients and doctors.
- Prescription stores medicines prescribed during an appointment.

We separate Prescription because:

- One appointment may include multiple medicines.
- Prescription information should not be repeated in the Appointment table.
- This reduces redundancy and makes the database easier to maintain**.**

3. Online Shopping System

Database tables include:

- Product(ProductID, ProductName, Price)
- Order(OrderID, CustomerID, OrderDate)
- OrderDetails(OrderID, ProductID, Quantity)

Each order may contain multiple products.

Answers:

1. Identification of Entities and Attributes

- Product(ProductID, ProductName, Price)
- Order(OrderID, CustomerID, OrderDate)
- OrderDetails(OrderID, ProductID, Quantity)

2. Understanding of Relationships

- One Order → Many Products
- One Product → Many Orders
- OrderDetails connects both.

3. Analysis of Keys and Constraints

Primary Keys

- ProductID
- OrderID
- (OrderID, ProductID)

Foreign Keys

- OrderID → Order
- ProductID → Product

4. Detection of Design Issues

- Without OrderDetails, only one product can be stored per order

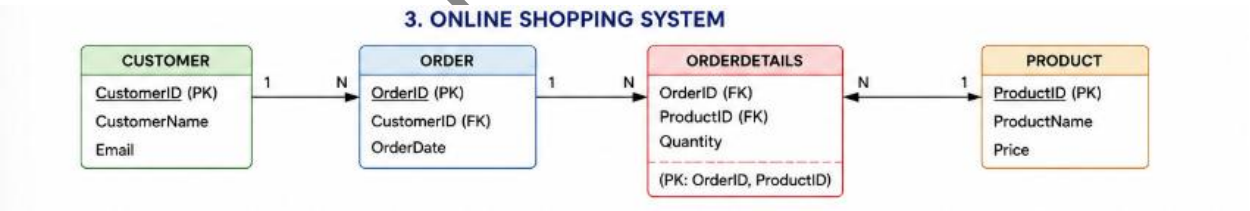
Viva Questions

Q1. Why is the OrderDetails table required in this schema?

- One order can contain many products.
- One product can appear in many orders.
- OrderDetails handles this many-to-many relationship and stores quantity.

Q2. What would happen if ProductID were stored directly in the Order table?

- Only one product could be stored per order.
- Multiple products cannot be handled.
- Data redundancy would increase.



Why this schema?

The database is divided into three tables.

- **Product** stores product information.
- **Order** stores order information.
- **OrderDetails** stores the products included in each order.

The **OrderDetails** table is necessary because:

- One order can contain many products.
- One product can appear in many orders.
- This is a **Many-to-Many (M:N)** relationship.

- The bridge table also stores the **Quantity** of each product in an order.

If ProductID were stored directly in the Order table, an order could contain only one product, which is incorrect for an online shopping system.